

Adaptive Memory Chunk eXtended (AMCX)

Documentación Técnica del Repositorio

1. Resumen del Proyecto

AMCX es una librería Python para almacenamiento persistente de memoria mediante archivos .amcx. Está orientada a sistemas de IA y conversación, permitiendo almacenar información en chunks comprimidos con mecanismos de integridad y recuperación.

2. Objetivos de Diseño

Reducir uso de memoria, permitir lectura selectiva, soportar compresión múltiple, verificar integridad mediante CRC32 y proporcionar recuperación mediante mirror SHA-1 y bloques XOR.

3. Estructura del Repositorio

amcx/: núcleo de la biblioteca. tests/: pruebas automatizadas. simple_example.py: ejemplo de integración. pyproject.toml: configuración del paquete.

4. Módulo format.py

Define constantes del formato binario, tamaños de cabecera, estructuras serializadas, tipos de chunk, algoritmos de compresión y flags del archivo.

5. Módulo compression.py

Implementa compresión y descompresión utilizando NONE, ZLIB y LZMA. También proporciona nombres legibles de algoritmos.

6. Módulo exceptions.py

Contiene excepciones específicas: archivo inválido, corrupción de datos, versión incompatible, chunk inexistente y errores de solo lectura.

7. Módulo writer.py

AMCXWriter construye archivos .amcx en memoria. Permite agregar ChunkEntry, comprimir contenido, calcular CRC32, generar índices y persistir archivos.

8. Clase ChunkEntry

Representa un chunk antes de ser serializado. Incluye identificador, tipo, resumen, contenido, algoritmo de compresión y timestamp.

9. Módulo reader.py

AMCXReader abre archivos .amcx, valida cabeceras, carga índices y recupera contenido bajo demanda verificando CRC32.

10. IndexEntry

Describe metadatos de cada chunk: offset, tamaño comprimido, tamaño original, algoritmo, timestamp, CRC32 y resumen.

11. SmartMemory

API de alto nivel. Automatiza segmentación de memoria, persistencia, búsqueda textual, caché y lectura selectiva.

12. Flujo SmartMemory

`append()` agrega mensajes; `flush()` genera chunks; `search()` busca texto; `get_recent()` recupera mensajes recientes; `verify_integrity()` valida consistencia.

13. Sistema Mirror

MirrorMode soporta NONE, MANUAL y AUTO. El modo AUTO incrusta hashes SHA-1 por chunk para detectar modificaciones.

14. Recuperación XOR

AMCXRecovery crea bloques de paridad XOR para recuperar información ante corrupción parcial.

15. Formato del Archivo

Cabecera AMC\0, versión, número de chunks, offset del índice y flags. Posteriormente se almacenan chunks comprimidos e índice estructurado.

16. Integridad

Cada chunk almacena CRC32. El lector recalcula el valor y detecta corrupción antes de devolver contenido.

17. Pruebas Automatizadas

`test_writer_reader.py` valida escritura y lectura. `test_mirror.py` valida mirror y recuperación. `test_smart.py` valida la API de alto nivel.

18. Ejemplo de Uso

SmartMemory permite crear memorias persistentes, almacenar conversaciones y realizar búsquedas contextuales para aplicaciones de IA.

19. Limitaciones Observadas

Existe una prueba marcada como TODO relacionada con `get_recent` y lógica de sincronización entre mensajes pendientes y persistidos.

20. Conclusiones

AMCX implementa una arquitectura modular que combina almacenamiento eficiente, compresión adaptativa e integridad verificable para escenarios de memoria persistente.